

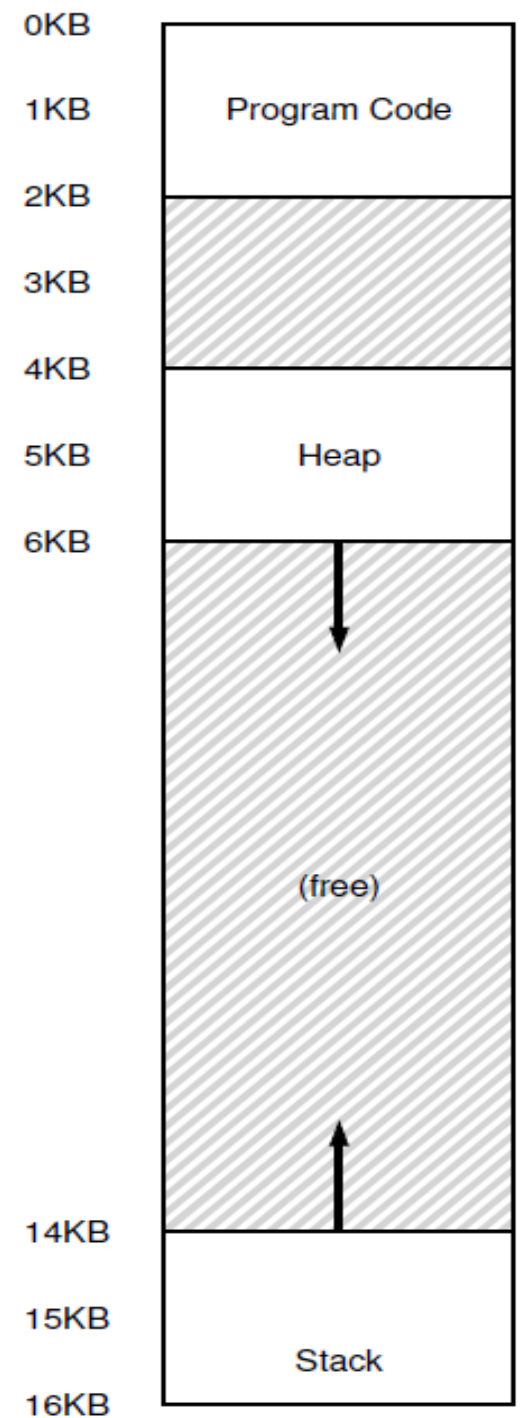
Memory Management: Segmentation

CS232 Operating Systems – Fall 2019

Taj Muhammad Khan

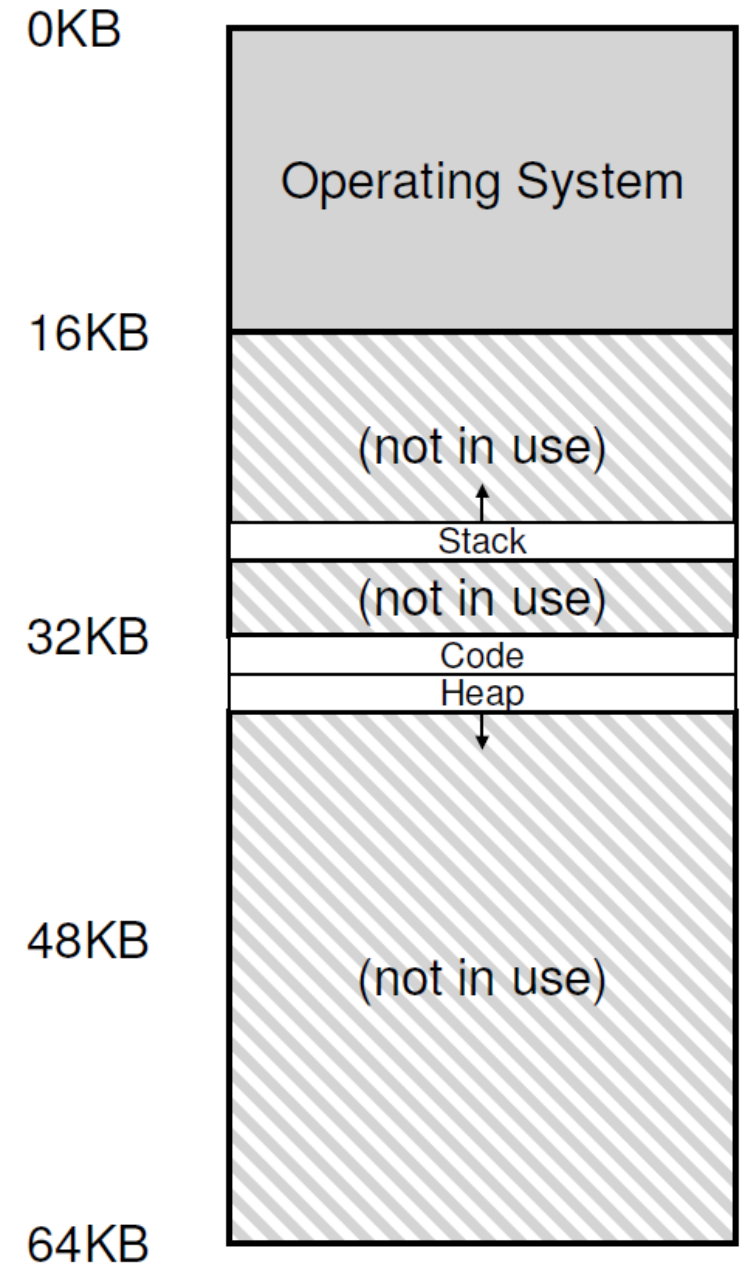
Address spaces in memory

- Usually process address spaces are sparsely populated
i.e. most of the space is empty
- Waste of RAM space
- What if a process address space is bigger than RAM?



Segmentation

- The address space consists of logical segments
- Instead of having one base/bound register pair, if we could have per-segment base/bound pairs?
- Each segment can then be placed independently in a separate part of physical memory!



Segmentation

- Saves memory
- Large address spaces can be accommodated
- Needs hardware support
 - Three base/bound pairs in this case

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Figure 16.3: **Segment Register Values**

Segmentation: example

- Process accesses:

- Virtual address 100 (code segment)
- Physical address = $32k + 100 = 32868$

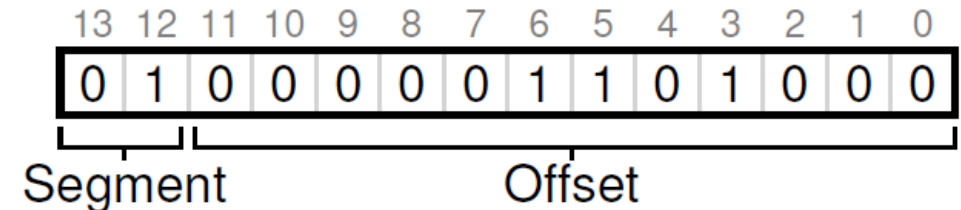
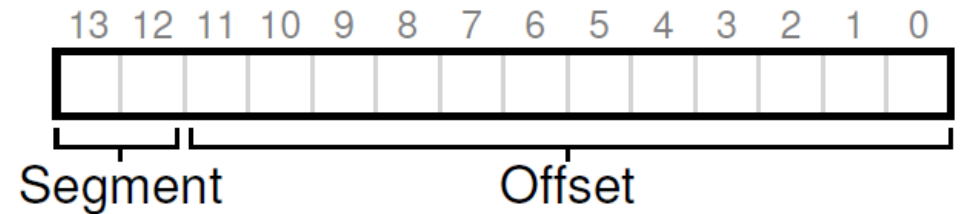
Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Figure 16.3: **Segment Register Values**

- Virtual address 4200 (heap segment)
- Physical address = $34k + 4200 = 39016$ (wrong!)
- We need to calculate heap offset: $\rightarrow 4200 - 4096 = 104$!
- Correct Physical address = $34k + 104 = 34920$
- What if we try to refer to an address outside the segment?
- Segmentation Fault!!

Which segment are we accessing

- How does the hardware know which base/bound pair to use?
- Explicit approach:
 - We know we have 3 segments
 - We can use the top few bits to identify the segment
 - If segments are placed at correct offset:
 - 00 -> code
 - 01 -> heap
 - 11 -> stack
- Calculating offset is also easier
 - VA → 4200



Which segment are we accessing... 2

- Explicit approach

```
1 // get top 2 bits of 14-bit VA
2 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3 // now get offset
4 Offset = VirtualAddress & OFFSET_MASK
5 if (Offset >= Bounds[Segment])
6     RaiseException(PROTECTION_FAULT)
7 else
8     PhysAddr = Base[Segment] + Offset
9     Register = AccessMemory(PhysAddr)
```

In our running example, we can fill in values for the constants above. Specifically, `SEG_MASK` would be set to `0x3000`, `SEG_SHIFT` to `12`, and `OFFSET_MASK` to `0xFFF`.

Which segment are we accessing... 3

- Implicit approach:
- Hardware determines the segment from the provenance of VA:
 - If the address is being generated via PC → Code segment
 - If the address is being generated via SP → Stack segment
 - Any thing else → Heap segment

What about the stack?

- It grows backwards!

• VA: 16KB – 14KB → PA: 28KB – 26KB

- Need hardware support

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

- Process addresses:

• VA: 15KB. 11 1100 0000 0000 (hex 0x3C00) VA offset = 3KB

• To obtain physical offset we've to subtract max segment size from this offset
i.e. 3KB – 4KB = -1 → Physical address = 28KB – 1KB = 27KB

Protection and Sharing

- Segmentation opens many doors

- With some HW support:

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

- Protection

- We can mark different segments as read only, writeable, executable

- Sharing

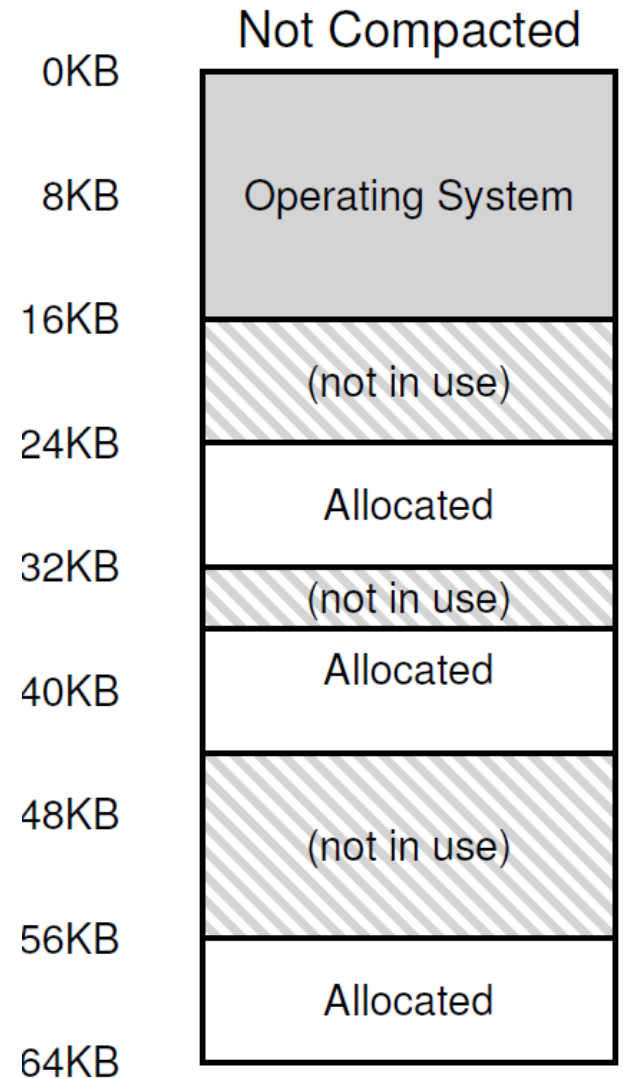
- Code segment can be marked read only and shared b/w processes
 - Libraries

Coarse grained vs Fine grained segmentation

- We only had 3 segments → coarse grained
- Some systems provide support for 1000s of segments
 - via Segment Tables

Segmentation: OS Support

- Set up the segment registers when a process runs
- Save and load segment registers on context switch
- Free space management
 - OS has to find space for segments of a new process
 - Segments may be of unequal sizes
 - Processes come and go
 - Memory may become full of small holes
 - This is called External Fragmentation



Segmentation: OS support

- Fragmentation can render RAM unusable
- Compaction is one solution
 - It is expensive

